

Porównanie wydajności CPU i GPU w przetwarzaniu obrazów

1. Cel projektu

Celem projektu było stworzenie i porównanie implementacji algorytmów przetwarzania obrazu wykonywanych na procesorze CPU oraz GPU z wykorzystaniem technologii CUDA i biblioteki Numba.

Projekt koncentruje się na:

- implementacji filtrów obrazu,
- wykorzystaniu równoległości GPU,
- optymalizacji obliczeń,
- analizie wydajności CPU i GPU,
- pomiarach czasu wykonania,
- analizie przyspieszenia speedup.

2. Technologie

Projekt został wykonany w języku Python z wykorzystaniem: Python, CUDA, Numba, NumPy, Matplotlib, Google Colab

2. Opis problemu

Przetwarzanie obrazów jest zadaniem wymagającym dużej liczby operacji wykonywanych na macierzach pikseli. Operacje takie jak:

- filtrowanie,
- rozmywanie,
- wykrywanie krawędzi

mogą być wykonywane niezależnie dla wielu pikseli jednocześnie, co czyni je bardzo dobrym kandydatem do akceleracji przy pomocy GPU.

W projekcie wykorzystano architekturę CUDA umożliwiającą wykonywanie obliczeń równoległych na tysiącach wątków GPU.

3. Model matematyczny

Projekt opiera się na matematycznym modelu cyfrowego przetwarzania obrazów wykorzystującym operację splotu (convolution). Obraz cyfrowy traktowany jest jako

dwuwymiarowa macierz wartości pikseli, gdzie każda wartość reprezentuje jasność danego punktu obrazu.

Dla obrazów w skali szarości każdy piksel przyjmuje wartość z zakresu:

$$0 \leq I(x,y) \leq 1$$

gdzie:

- $I(x,y)$ oznacza wartość jasności piksela,
- x,y są współrzędnymi piksela w obrazie.

W projekcie zastosowano lokalne operacje filtrowania obrazu. Oznacza to, że nowa wartość każdego piksela obliczana jest na podstawie jego najbliższego sąsiedztwa. Wykorzystywane jest okno o rozmiarze 3x3, obejmujące analizowany piksel oraz jego ośmiu sąsiadów.

Podstawową operacją matematyczną wykorzystywaną w projekcie jest splot, czyli convolution. Operacja ta polega na przesuwaniu specjalnej maski tj. kernelu po obrazie i wykonywaniu mnożenia odpowiadających sobie elementów, a następnie sumowaniu wyników.

Ogólny wzór operacji splotu:

$$I'(x,y) = \sum_{i=-k}^k \sum_{j=-k}^k I(x+i, y+j) \cdot K(i,j)$$

gdzie:

- $I(x,y)$ - obraz wejściowy,
- $I'(x,y)$ - obraz wynikowy,
- $K(i,j)$ - maska filtru (kernel),
- k - połowa rozmiaru maski.

W projekcie zastosowano trzy rodzaje filtrów: Mean Filter, Gaussian Blur, Sobel Edge Detection.

4. Wykorzystane algorytmy

4.1 Mean Filter

Filtr uśredniający oblicza średnią wartość pikseli znajdujących się w sąsiedztwie analizowanego piksela. Powoduje to wygładzanie obrazu oraz redukcję szumu. Nowa wartość piksela wyznaczana jest jako średnia arytmetyczna wartości wszystkich pikseli w oknie 3x3.

Wzór:

$$I'(x, y) = \frac{1}{9} \sum_{i=-1}^1 \sum_{j=-1}^1 I(x+i, y+j)$$

Maska filtru:

$$K = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

4.2 Gaussian Blur

Gaussian Blur wykorzystuje filtr oparty na rozkładzie Gaussa. W przeciwieństwie do mean filter większe znaczenie przypisywane jest pikselom znajdującym się bliżej środka maski. Filtr ten zapewnia bardziej naturalne rozmycie obrazu i lepiej zachowuje jego strukturę.

Maska filtru:

$$K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Filtr ten zachowuje więcej szczegółów niż prosty mean filter.

4.3 Sobel Edge Detection

Operator Sobela służy do wykrywania krawędzi poprzez obliczanie gradientu jasności obrazu.

Wykorzystuje dwa filtry:

Gradient poziomy:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Gradient pionowy:

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Dla każdego piksela obliczane są dwa gradienty: poziomy, pionowy.

Następnie obliczana jest końcowa wartość gradientu:

$$G = \sqrt{G_x^2 + G_y^2}$$

Duże wartości gradientu oznaczają występowanie krawędzi obrazu.

6. Architektura CUDA

CUDA jest platformą programistyczną firmy NVIDIA umożliwiającą wykonywanie obliczeń równoległych na kartach graficznych GPU. W przeciwieństwie do procesora CPU, który posiada niewielką liczbę bardzo wydajnych rdzeni, GPU zawiera tysiące prostszych rdzeni zoptymalizowanych do równoczesnego wykonywania dużej liczby operacji matematycznych.

Przetwarzanie obrazów jest bardzo dobrym zastosowaniem dla GPU, ponieważ operacje wykonywane na pikselach są w dużym stopniu niezależne od siebie i mogą być wykonywane równolegle.

6.1 Model wykonywania CUDA

CUDA wykorzystuje hierarchiczny model wykonywania:

- wątki (threads),
- bloki (blocks),
- siatki bloków (grid).

6.2 Threads

Najmniejszą jednostką wykonywania w CUDA jest wątek. W projekcie jeden wątek GPU odpowiada za przetworzenie jednego piksela obrazu.

Każdy wątek oblicza wartość filtru, rozmycie lub wykrywanie krawędzi dla pojedynczego punktu obrazu.

Identyfikator wątku obliczany jest przy pomocy: `row, col = cuda.grid(2)`

gdzie:

- `row` - numer wiersza,
- `col` - numer kolumny.

Dzięki temu każdy wątek zna pozycję piksela, który ma przetworzyć.

6.3 Blocks

Wątki grupowane są w bloki (thread blocks).

W projekcie wykorzystano konfigurację: `threads = (16,16)`

co oznacza: $16 * 16 = 256$ wątków na blok.

Taki rozmiar bloku:

- jest zgodny z architekturą CUDA,
- zapewnia dobrą wydajność,
- dobrze wykorzystuje zasoby GPU.

6.4 Grid

Bloki tworzą siatkę, która pokrywa cały obraz. Liczba bloków wyznaczana była dynamicznie:

```
blocks = (  
    (width + 15) // 16,  
    (height + 15) // 16  
)
```

Dzięki temu każdy piksel obrazu zostaje przypisany do jednego wątku oraz możliwe jest równoległe przetwarzanie całego obrazu.

6.5 Równoległość GPU

GPU wykonuje tysiące wątków jednocześnie.

Dla obrazu: 4096×4096 , oznacza to ponad: 16 777 216 pikseli przetwarzanych równoległe. To właśnie równoległość GPU odpowiada za znaczące przyspieszenie względem CPU.

7. Przygotowanie obrazu wejściowego

Obraz wejściowy został wczytany przy pomocy biblioteki Pillow oraz przekonwertowany do skali szarości. Następnie wartości pikseli zostały znormalizowane do zakresu $[0,1]$, co upraszcza wykonywanie obliczeń numerycznych.

Przykładowy fragment kodu:

```
img = Image.open("house.jpg").convert("L")
img = np.array(img).astype(np.float32) / 255.0
```

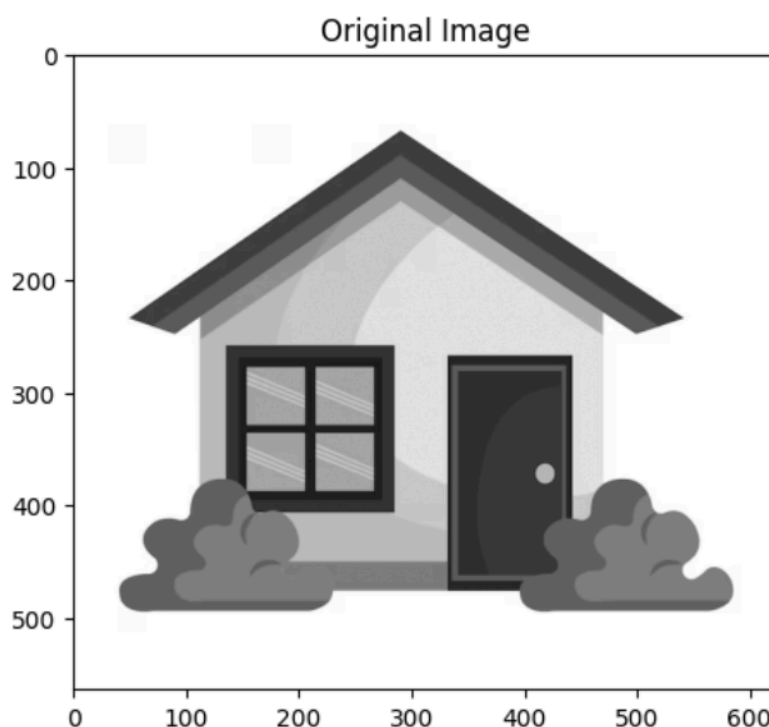
Konwersja: `convert("L")`

powoduje przejście obrazu do skali szarości, to znaczy grayscale, dzięki czemu każdy piksel reprezentowany jest pojedynczą wartością jasności.

Normalizacja: `/ 255.0`

zamienia wartości pikseli z zakresu: 0-255, na: 0-1

co poprawia stabilność obliczeń i upraszcza implementację filtrów.



Rys.1. Obraz wejściowy w skali szarości

8. Mean Filter - implementacja CPU i GPU

Filtr uśredniający został zaimplementowany zarówno dla procesora CPU, jak i dla GPU. Celem algorytmu jest wygładzenie obrazu poprzez obliczenie średniej wartości pikseli w lokalnym oknie 3×3 .

Implementacja CPU

Wersja CPU została zaimplementowana przy pomocy biblioteki Numba oraz równoległych pętli prange.

Fragment implementacji:

```
@jit(parallel=True)
def mean_filter_cpu(image):

    rows, cols = image.shape
    result = np.zeros_like(image)

    for i in prange(1, rows - 1):
        for j in range(1, cols - 1):

            acc = 0.0

            for r in range(-1, 2):
                for c in range(-1, 2):

                    acc += image[i + r, j + c]

            result[i, j] = acc / 9.0

    return result
```

Algorytm iteruje po wszystkich pikselach obrazu, pobiera sąsiedztwo 3×3, sumuje wartości pikseli oraz oblicza średnią arytmetyczną.

Dekorator: @jit(parallel=True)
umożliwia kompilację Just-In-Time i częściową równoległość CPU.

Implementacja GPU

Wersja GPU została zaimplementowana jako kernel CUDA.

Fragment implementacji:

```
@cuda.jit
def mean_filter_gpu(image, result):

    row, col = cuda.grid(2)

    rows, cols = image.shape

    if 0 < row < rows - 1 and 0 < col < cols - 1:

        acc = 0.0

        for r in range(-1, 2):
            for c in range(-1, 2):

                acc += image[row + r, col + c]

        result[row, col] = acc / 9.0
```

Każdy wątek GPU odpowiada za pojedynczy piksel obrazu i wykonuje operację filtrowania niezależnie od innych wątków.

Instrukcja: `row, col = cuda.grid(2)`

oblicza współrzędne piksela przypisanego do danego wątku CUDA.

Porównanie CPU i GPU

Wersja CPU:

- wykorzystuje ograniczoną liczbę rdzeni procesora,
- wykonuje obliczenia częściowo równolegle.

Wersja GPU:

- wykorzystuje tysiące wątków CUDA,
- przetwarza wiele pikseli jednocześnie,
- osiąga znacznie większą wydajność dla dużych obrazów.

8. Gaussian Blur - implementacja CPU i GPU

Gaussian Blur jest filtrem wygładzającym wykorzystywanym w przetwarzaniu obrazów do redukcji szumu, wygładzania obrazu i usuwania drobnych szczegółów.

W przeciwieństwie do prostego filtru uśredniającego większą wagę przypisuje się pikselom znajdującym się bliżej środka maski.

Implementacja CPU

Wersja CPU została zaimplementowana przy pomocy biblioteki Numba oraz równoległych pętli prange.

Fragment implementacji:

```
@jit(parallel=True)
def gaussian_blur_cpu(image):

    rows, cols = image.shape
    result = np.zeros_like(image)

    kernel = np.array([
        [1,2,1],
        [2,4,2],
        [1,2,1]
    ], dtype=np.float32) / 16.0

    for i in prange(1, rows-1):

        for j in range(1, cols-1):

            acc = 0.0

            for r in range(-1,2):
                for c in range(-1,2):

                    acc += (
                        image[i+r, j+c]
                        * kernel[r+1, c+1]
                    )

            result[i,j] = acc
    return result
```

Algorytm:

- iteruje po wszystkich pikselach obrazu,
- pobiera lokalne sąsiedztwo 3×3 ,
- mnoży wartości pikseli przez współczynniki maski,
- sumuje wyniki,
- zapisuje wynik do obrazu wyjściowego.

Implementacja GPU

Wersja GPU została zaimplementowana jako kernel CUDA wykonywany równolegle przez tysiące wątków GPU.

Fragment implementacji:

```
@cuda.jit
def gaussian_blur_gpu(image, result):

    row, col = cuda.grid(2)

    rows, cols = image.shape

    if 0 < row < rows-1 and 0 < col < cols-1:

        acc = 0.0

        acc += image[row-1,col-1] * 1
        acc += image[row-1,col] * 2
        acc += image[row-1,col+1] * 1

        acc += image[row,col-1] * 2
        acc += image[row,col] * 4
        acc += image[row,col+1] * 2

        acc += image[row+1,col-1] * 1
        acc += image[row+1,col] * 2
        acc += image[row+1,col+1] * 1

        result[row,col] = acc / 16.0
```

Optymalizacja GPU

Implementacja GPU:

- eliminuje konieczność sekwencyjnego przetwarzania obrazu,
- wykorzystuje masową równoległość GPU,
- umożliwia jednoczesne przetwarzanie milionów pikseli.

Dzięki temu:

- czas wykonania GPU jest znacznie krótszy niż CPU,
- szczególnie dla dużych obrazów.

9. Sobel Edge Detection - implementacja CPU i GPU

Operator Sobela jest jednym z najpopularniejszych algorytmów wykrywania krawędzi w przetwarzaniu obrazów.

Celem algorytmu jest:

- wykrywanie gwałtownych zmian jasności,
- identyfikacja konturów oraz krawędzi obiektów znajdujących się na obrazie.

Algorytm oblicza gradient jasności obrazu w dwóch kierunkach:

- poziomym Gx,
- pionowym Gy.

Implementacja CPU

Fragment implementacji:

```
@njit(parallel=True)
def sobel_cpu(image):

    rows, cols = image.shape
    result = np.zeros_like(image)

    Gx = np.array([
        [-1,0,1],
        [-2,0,2],
        [-1,0,1]
    ], dtype=np.float32)

    Gy = np.array([
```

```

        [-1,-2,-1],
        [0,0,0],
        [1,2,1]
    ], dtype=np.float32)

    for i in prange(1, rows-1):

        for j in range(1, cols-1):

            sx = 0.0
            sy = 0.0

            for r in range(-1,2):
                for c in range(-1,2):

                    pixel = image[i+r, j+c]

                    sx += pixel * Gx[r+1,c+1]
                    sy += pixel * Gy[r+1,c+1]

            result[i,j] = np.sqrt(sx*sx + sy*sy)

    return result

```

Algorytm:

- pobiera lokalne sąsiedztwo piksela,
- oblicza gradient poziomy,
- oblicza gradient pionowy,
- wyznacza końcową wartość gradientu.

Implementacja GPU

Wersja GPU została zaimplementowana jako kernel CUDA wykonywany równolegle przez tysiące wątków GPU.

Fragment implementacji:

```
@cuda.jit
def sobel_gpu(image, result):

    row, col = cuda.grid(2)

    rows, cols = image.shape

    if 0 < row < rows-1 and 0 < col < cols-1:

        sx = 0.0
        sy = 0.0

        # Gx
        sx += -1 * image[row-1,col-1]
        sx += 1 * image[row-1,col+1]

        sx += -2 * image[row,col-1]
        sx += 2 * image[row,col+1]

        sx += -1 * image[row+1,col-1]
        sx += 1 * image[row+1,col+1]

        # Gy
        sy += -1 * image[row-1,col-1]
        sy += -2 * image[row-1,col]
        sy += -1 * image[row-1,col+1]

        sy += 1 * image[row+1,col-1]
        sy += 2 * image[row+1,col]
        sy += 1 * image[row+1,col+1]

        result[row,col] = (sx*sx + sy*sy)**0.5
```

Każdy wątek GPU:

- odpowiada za pojedynczy piksel obrazu,
- oblicza gradient poziomy i pionowy,
- wyznacza wartość krawędzi.

Wykrywanie krawędzi

Operator Sobela działa na zasadzie wykrywania dużych zmian jasności pomiędzy sąsiednimi pikselami. Obszary o małych zmianach jasności, posiadają małe wartości gradientu. Natomiast granice obiektów, kontury, ostre przejścia powodują wysokie wartości gradientu i są widoczne jako jasne krawędzie na obrazie wynikowym.

10. Konfiguracja CUDA

W celu uruchomienia kerneli CUDA konieczne było zdefiniowanie konfiguracji wykonywania obejmującej:

- liczbę wątków w bloku,
- liczbę bloków w siatce.

W projekcie zastosowano następującą konfigurację:

```
threads = (16,16)
blocks = (
    (img.shape[1] + 15)//16,
    (img.shape[0] + 15)//16)
```

- Threads per block oznacza, że każdy blok CUDA zawiera $16 \times 16 = 256$ wątków.

Układ 16×16 :

- dobrze odpowiada dwuwymiarowej strukturze obrazu,
- zapewnia efektywne wykorzystanie architektury GPU,
- jest często stosowaną konfiguracją w przetwarzaniu obrazów.

Każdy wątek odpowiada za przetworzenie jednego piksela obrazu.

- Grid size to Liczba bloków obliczana jest dynamicznie na podstawie rozmiaru obrazu.

Gdzie:

- `img.shape[1]` - szerokość obrazu,
- `img.shape[0]` - wysokość obrazu.

Dodanie wartości: +15 umożliwia poprawne zaokrąglenie liczby bloków w górę. Dzięki temu wszystkie piksele obrazu zostają pokryte siatką CUDA oraz również wtedy, gdy rozmiar obrazu nie jest wielokrotnością 16.

11. Benchmark i pomiar wydajności

W projekcie wykonano pomiary czasu działania algorytmów: Mean Filter, Gaussian Blur i Sobel Edge Detection dla CPU i GPU. Celem benchmarków było porównanie wydajności obliczeń sekwencyjnych i równoległych.

11.1 Pomiar czasu CPU

Czas wykonania CPU mierzony był przy pomocy funkcji: `perf_counter_ns()`

Przykład:

```
t0 = perf_counter_ns()
mean_cpu = mean_filter_cpu(img)
t1 = perf_counter_ns()
```

Następnie czas przeliczano na milisekundy: `mean_cpu_time = (t1 - t0) / 1e6`

11.2 Warmup JIT

Przed właściwym pomiarem wykonywano dodatkowe uruchomienie funkcji:

```
dummy = np.random.rand(32,32).astype(np.float32)
mean_filter_cpu(dummy)
```

Pozwala to na:

- wcześniejszą kompilację JIT przez Numba,
- uniknięcie zawyżenia pierwszego pomiaru.

Pierwsze wywołanie funkcji Numba obejmuje proces kompilacji, dlatego nie powinno być uwzględniane w benchmarku.

11.3 Pomiar czasu GPU

Czas wykonania GPU mierzony był przy pomocy zdarzeń CUDA.

Przykład:

```
start = cuda.event()
end = cuda.event()

start.record()

mean_filter_gpu[blocks, threads](d_img, d_res)

end.record()
end.synchronize()
```

Czas kernela obliczano przy pomocy: `cuda.event_elapsed_time(start, end)`

11.4 Synchronizacja GPU

Po uruchomieniu kernela stosowano: `cuda.synchronize()`

Synchronizacja jest konieczna, ponieważ operacje GPU wykonywane są asynchronicznie. CPU może kontynuować działanie przed zakończeniem kernela CUDA. Bez synchronizacji pomiar czasu byłby niepoprawny.

11.5 Transfer danych

Dane wejściowe kopiowane były do pamięci GPU: `d_img = cuda.to_device(img)`

Wynik kopiowany był z powrotem do pamięci CPU:

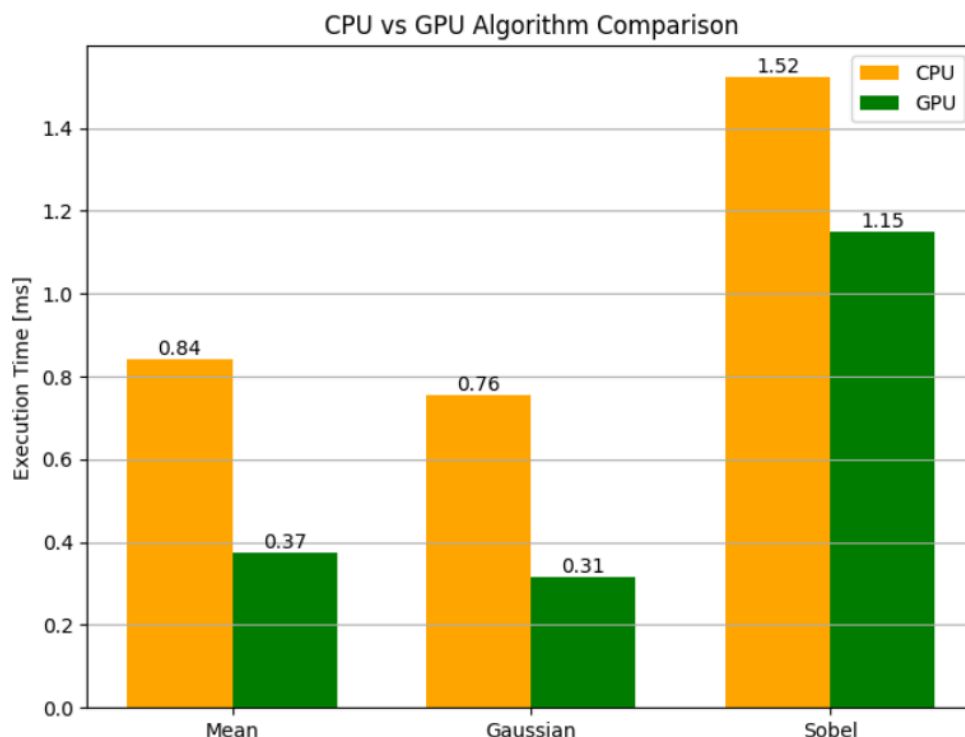
```
mean_gpu = d_res.copy_to_host()
```

11.6 Wyniki i analiza wydajności

Tabela przedstawia porównanie czasu wykonania algorytmów Mean Filter, Gaussian Blur oraz Sobel Edge Detection.

Tabela 1. Czasy wykonania algorytmów.

Algorytm	CPU [ms]	GPU [ms]
Mean Filter	0.843	0.375
Gaussian Blur	0.757	0.315
Sobel Edge Detection	1.522	1.149



Rys. 2. Porównanie czasu wykonania algorytmów na CPU i GPU.

Wykres przedstawia różnice wydajności pomiędzy implementacją CPU i GPU dla analizowanych algorytmów przetwarzania obrazu. We wszystkich przypadkach implementacja GPU osiągała znacznie krótszy czas wykonania.

11.7 Podsumowanie

Największe przyspieszenie uzyskano dla algorytmu Gaussian Blur, a następnie dla Mean Filter. Algorytm Sobel Edge Detection wymaga większej liczby operacji matematycznych, dlatego jego czas wykonania jest dłuższy zarówno na CPU, jak i GPU. Równoległość GPU pozwala osiągnąć wyższą wydajność niż klasyczne wykonanie na CPU.

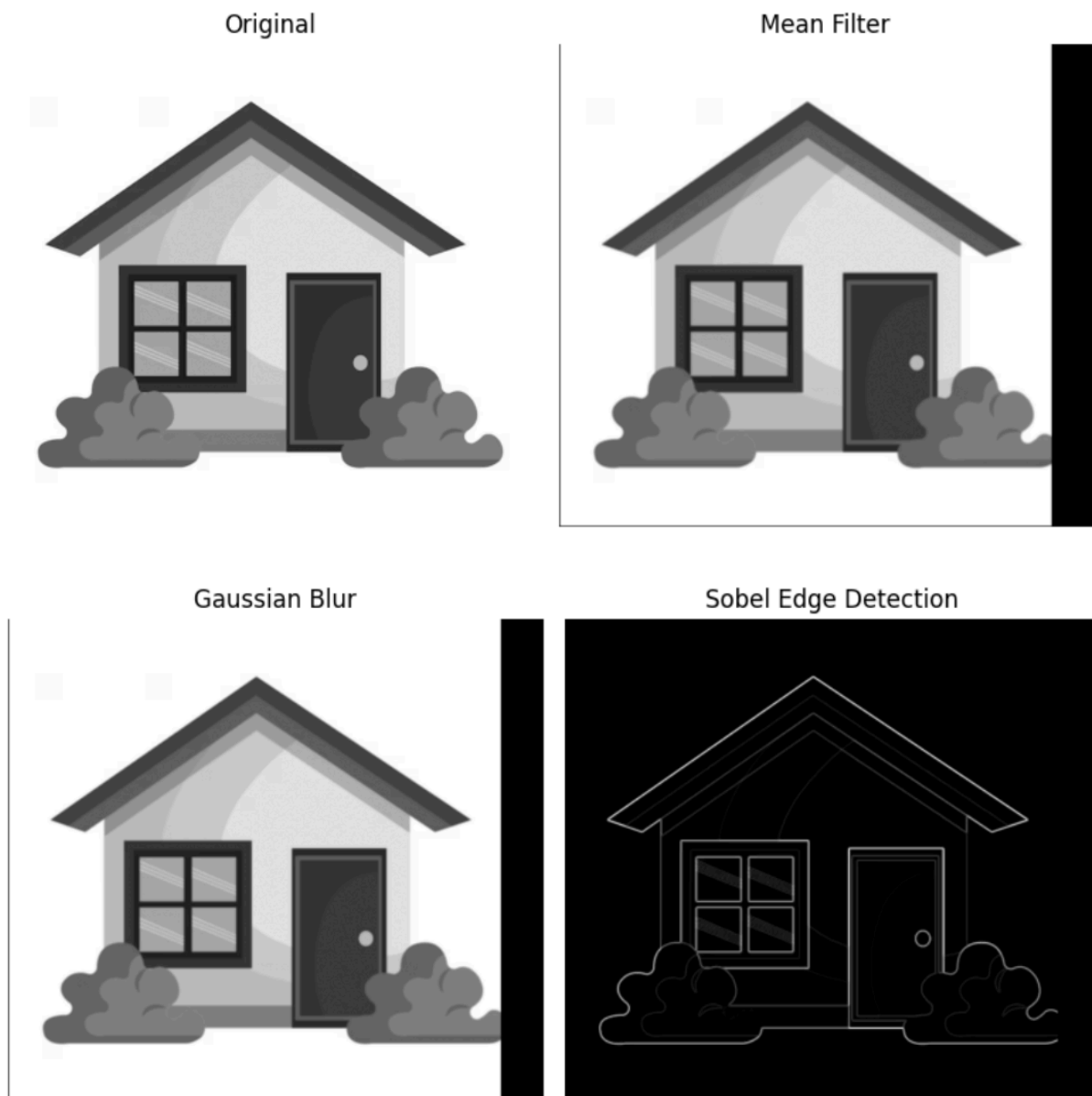
12. Wizualizacja wyników algorytmów

Na poniższym rysunku przedstawiono rezultaty działania zaimplementowanych algorytmów przetwarzania obrazu. Porównano obraz oryginalny z obrazami przetworzonymi przy pomocy GPU.

Każdy algorytm realizuje inne zadanie:

- Mean Filter wygładza obraz poprzez uśrednianie sąsiednich pikseli,

- Gaussian Blur redukuje szумы oraz wygładza obraz przy pomocy filtru Gaussa,
- Sobel Edge Detection wykrywa krawędzie obiektów na obrazie.



Rys. 3. Wizualizacja zastosowanych algorytmów.

- Original - oryginalny obraz wejściowy w skali szarości.
- Mean Filter - obraz po zastosowaniu filtru uśredniającego. Widoczne jest lekkie wygładzenie, redukcja drobnych szczegółów, zmniejszenie szumu.
- Gaussian Blur - obraz po zastosowaniu filtru Gaussa. W porównaniu do Mean Filter wygładzenie jest bardziej naturalne, zachowane zostają większe struktury obrazu, szum jest skuteczniej redukowany.
- Sobel Edge Detection - obraz po detekcji krawędzi. Widoczne są kontury obiektów, granice pomiędzy obszarami o różnej jasności,

12.1 Obsługa pikseli brzegowych

Podczas analizy wyników można zauważyć ciemne obszary przy krawędziach obrazów wynikowych. Nie jest to błąd algorytmu, lecz efekt zastosowanego mechanizmu sprawdzania granic (*boundary check*) w kernelach CUDA.

W implementacji zastosowano warunek:

```
if 0 < row < rows-1 and 0 < col < cols-1:
```

który powoduje pominięcie pikseli znajdujących się na krawędziach obrazu.

Dodatkowo:

- siatka CUDA może nie pokrywać idealnie całego obrazu,
- część pikseli przy krawędzi może pozostać niezapisana,
- niezapisane wartości przyjmują wartość 0.

W obrazie grayscale wartość: 0

oznacza kolor czarny, dlatego widoczne są ciemne paski przy krawędziach.

Takie uproszczenie implementacji jest często stosowane w algorytmach convolution wykonywanych na GPU, ponieważ pozwala uniknąć odwołań poza zakres tablicy.

13. Wpływ rozmiaru obrazu na wydajność

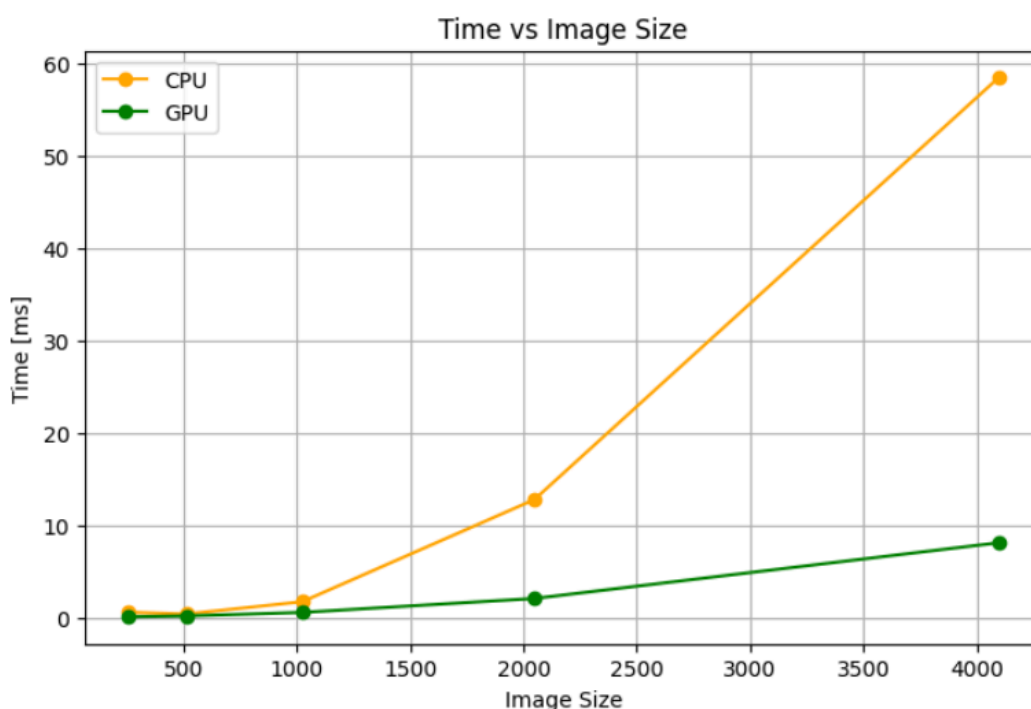
W celu analizy skalowalności implementacji GPU wykonano pomiary czasu działania **algorytmu Gaussian Blur** dla różnych rozmiarów obrazów. Porównano czas wykonania implementacji CPU i GPU.

Testy przeprowadzono dla obrazów 256×256, 512×512, 1024×1024, 2048×2048 i 4096×4096.

Tabela 2. Czas dla różnych rozdzielczości obrazu.

Rozmiar obrazu	CPU [ms]	GPU [ms]
256×256	0.61	0.11
512×512	0.42	0.21
1024×1024	1.75	0.59
2048×2048	12.81	2.10
4096×4096	58.49	8.13

Tabela przedstawia czas wykonania algorytmu Gaussian Blur dla różnych rozmiarów obrazów.



Rys. 4. Zależność czasu wykonania od rozmiaru obrazu.

Wykres przedstawia zależność czasu wykonania od rozmiaru obrazu dla implementacji CPU oraz GPU.

Wnioski

Przeprowadzone testy pokazują, że wraz ze wzrostem rozmiaru obrazu rośnie przewaga GPU nad CPU. Dla małych obrazów, takich jak 256×256 , 512×512 różnica pomiędzy CPU i GPU jest stosunkowo niewielka. Wynika to z narzutu związanego z uruchomieniem kernela CUDA oraz transferem danych do pamięci GPU. W przypadku większych obrazów przewaga GPU staje się znacznie bardziej widoczna dla obrazu 4096×4096 .

Uzyskane wyniki pokazują, że GPU bardzo dobrze skaluje się wraz ze wzrostem liczby danych. Operacje convolution są szczególnie efektywne na architekturze równoległej.

15. Analiza przyspieszenia GPU Speedup

W celu określenia efektywności akceleracji GPU obliczono współczynnik przyspieszenia, czyli speedup dla różnych rozmiarów obrazów, wykorzystując **algorytm Gaussian Blur**.

Współczynnik speedup definiowany jest jako:

$$Speedup = \frac{T_{CPU}}{T_{GPU}}$$

gdzie:

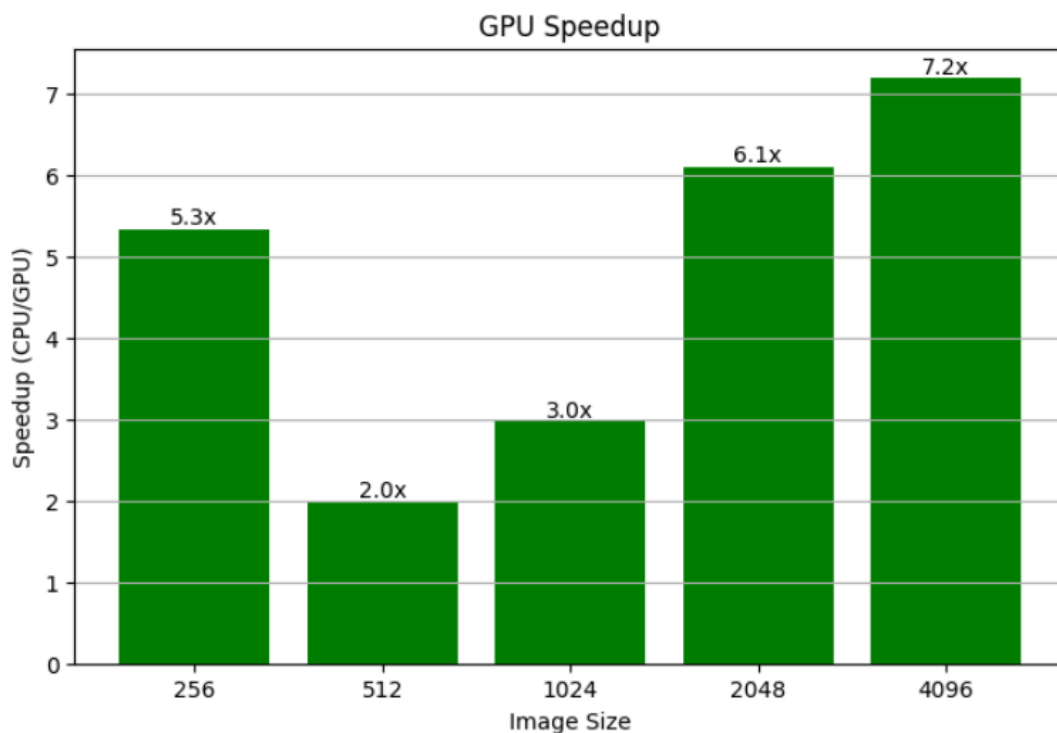
- T_{CPU} - czas wykonania na CPU,
- T_{GPU} - czas wykonania na GPU.

Interpretacja

Wartość:

- 1x oznacza brak przyspieszenia,
- większe wartości oznaczają przewagę GPU.

Przykładowo: 5x oznacza, że GPU działa pięć razy szybciej niż CPU.



Rys. 5. Współczynnik przyspieszenia GPU względem CPU

Wykres przedstawia współczynnik przyspieszenia GPU względem CPU dla różnych rozmiarów obrazów.

Analiza wyników

Na podstawie uzyskanych wyników można zauważyć, że wraz ze wzrostem rozdzielczości obrazu rośnie efektywność GPU, w szczególności dla obrazu o rozmiarze 4096×4096, gdzie przyspieszenie osiągnęło około 7.2x względem CPU. Dla małych obrazów przyspieszenie jest mniejsze, ponieważ narzut związany z uruchomieniem kernela CUDA i transfer danych między CPU i GPU stanowią istotną część całkowitego czasu wykonania.

Otrzymane rezultaty potwierdzają, że:

- algorytmy przetwarzania obrazu bardzo dobrze skalują się na GPU,
- równoległość CUDA pozwala znacznie skrócić czas obliczeń,
- przewaga GPU rośnie wraz ze wzrostem ilości danych wejściowych.

16. Podsumowanie

Celem projektu było zaimplementowanie oraz porównanie algorytmów przetwarzania obrazu wykonywanych na CPU oraz GPU z wykorzystaniem technologii CUDA i biblioteki Numba.

W projekcie zaimplementowano Mean Filter, Gaussian Blur i Sobel Edge Detection.

Dla każdego algorytmu przygotowano:

- implementację CPU,
- implementację GPU,
- benchmark wydajności,
- analizę skalowalności.

Przeprowadzone testy pokazały, że GPU osiąga znacznie krótsze czasy wykonania niż CPU. Największe przyspieszenie występuje dla dużych obrazów. Algorytmy przetwarzania obrazu bardzo dobrze nadają się do równoległego wykonywania. Wraz ze wzrostem rozmiaru obrazu przewaga GPU staje się coraz większa, a CPU skaluje się znacznie gorzej niż GPU.

BIBLIOGRAFIA

- <https://numba.readthedocs.io/>
- <https://medium.com/@wilson.linzhe/digital-image-processing-in-c-chapter-1-mean-and-median-filter-b4c4d0775e14>
- <https://bioimagebook.github.io/chapters/2-processing/4-filters/filters.html>
- <https://medium.com/@deepika.vadlamudi/implementing-a-sobel-filter-with-cuda-in-python-2b9b18485e31>

- <https://arxiv.org/pdf/2305.00515>
- <https://fastcompression.medium.com/gpu-vs-cpu-at-image-processing-why-gpu-is-much-faster-than-cpu-f6a3a7fa2907>